Compute Platform for AI

An Open-Source Summary

Thomas Debelle

KU LEUVEN

October 10, 2025

Contents

1 Lecture 1: Towards heterogeneous many-core processors

1.1 Scaling

In IC and chip design, there is two fondamental laws:

- **Denard's law:** as transistors get smaller, the power density remains constant, which leads to lower and lower supply voltage to avoid to break down the oxide due to a strong $\vec{E}$.
- **Moore's law:** every generation can fit twice as many transistors on a certain area.

1.1.1 Denard broke down

Denard's law is based on the fact that the width, length, oxide thickness and voltage is reduced by a factor α each time. This factor also influences:

- Density: α^2
- Capacitance: $1/\alpha$
- Delay: $1/\alpha$

Thus, Power = $CV_{DD}^2 f \propto 1/\alpha \cdot cst \cdot 1/\alpha = 1/\alpha^2$. Finally the power density is a constant.

On paper, this is valid but we can't infinitely scale down V or we will have lower speed the closer we come to V_{th} which is not feasible. Moreover, the wire cannot scale down as desired or we will have a bad resistance in the wire. This will make the wires a bit more "capacitive" and so the α factors will no longer cross out as Denard predicted. The power is constant and the power density is α^2 which is quite problematic.

1.1.2 Dark and Dim Silicon

The end of the Denard's scaling lead to a plateau in the power consumption of chips. We have to buy this energy efficiency. We know that the power density scales with α^2 at maximum clock frequency. So we will introduce:

- **Dim silicon:** silicon running at below max f_ϕ/α^2
- **Dark silicon:** $1/\alpha^2$ blocks that totally shut down when not used

In practice, if we scale with a factor $S = 2$ we can put 2^2 more silicon and the speed should increase by a factor 2. In *dark silicon*, we will still speedup the clock frequency but all the newly added cores will be shutdown. This is quite unsustainable as more cores (due to Adhalm's law) won't leverage from parallelism. The better idea is to not clock faster and use this extra factor 2 to produce dim silicon and then the rest with dark silicon.

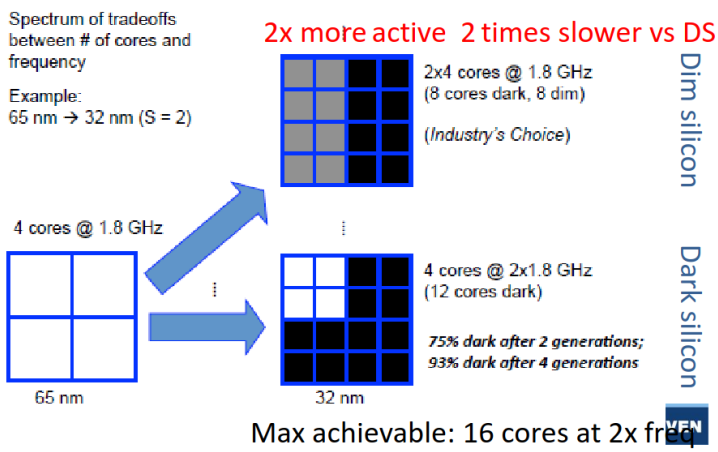

Figure 1: The two aforementioned techniques

Recently, we are using more and more dark silicon as accelerators that get turned on for specific task – accelerators.

1.2 Area for energy in single-core

To make a processor faster, we can use either:

- *Instruction-level parallelism*: VLIW, OOO super-scalar
- *Data-level parallelism*: SIMD, GPU

> Won't re-explain what those are, check the computer architecture lecture: *link*

Those are prime examples of dim silicon with lower clock speed for same throughput thanks to parallelism.

There is also the vector extension that can be seen as a sort of *temporal* data parallelism. SIMD, VLIW and other processing is often found in DSP chip since such processing is often data intensive.

1.2.1 Super-scalar

In this version, it is out of order and the processor can schedule instructions when it wants. It uses the Tomasulo's algorithm but must commit in order to avoid data dependency issues.

The advantage of such processor is the flexibility of the operations. Typically, some execution stage can take more time than others. We must use a reorder buffer (ROB) to commit in order. This is a prime example of spending extra transistors instead of clocking faster. There is a certain overhead for controlling such processors but the gains are far superior.

1.2.2 Memory area

Memory access time is the usual bottleneck in computing, a lot of time, the thread is blocked because it waits for data to arrive. Latency of cache and memory is quite important and to hide it we use multi-threading with some level of granularity to keep all cores as busy as possible.

We can't use too much of simultaneous multithreading SMT because the overhead is quite important. Need each time a program counter, register file and a reorder per thread. On top of that, the commit and scoreboard is more complex.

This is why we use various cache level and we exchange latency with transistors count. A recent trend is using large reorder buffer to see well in advance and to re-order online the instructions. This will increase parallelism and reduce off-chip memory accesses.

1.3 Area for energy through multi-core

Sadly, this is not enough and *Amdahl's law* explain that parallelism will not always lead to a speedup and lots of applications are not easily parallelizable.

1.3.1 Homogenous

We first started by doing `ctrl`+`c` and `ctrl`+`v` on the cores to realize a speedup. On top of that, we would some p-state (dim silicon) or shut it down completely (dark silicon) if not used.

1.3.2 Heterogeneous

Here, we will have dedicated low-power cores that can realize operations if low performance is needed. Depending on the workload, we can switch to efficiency core or performance core. This is present in the **big.LITTLE** ARM architecture. We must use the same instruction set to seamlessly transfer from one core to the other.

This idea is heavily used in embedded devices like smartphone and is starting to make its way through in personal computer, typically Apple's computer.

1.4 Area for energy through domain specific accelerators (DSA)

We are now facing the utilization wall. More and more silicon must be dim or dark, we must use the extra transistors for something. Here comes accelerators which are custom ASIC blocks for specific function.

2 Lecture 2: ISP's, GPU's and AI Workloads

2.1 ISP's & GPU's for graphics/images

2.1.1 ISP applications, architecture and operation

To take a picture, many steps are required: calibration, RGB conversion, encoding, post-processing, … Image Signal Processor is one of the earliest example of ASIC. The operations are fixed and easily paralleziable.

Initially, it was mostly hardwired but more recently introduction of more flexibility. It gained popularity in commercially available ISP as people demanded more from their smartphones and cameras.

2.1.1.1 IPU This is becoming more and more like an Image Processing Unit where we want the best of both world:

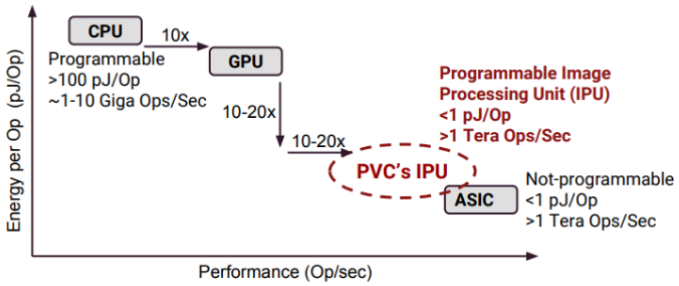

Figure 2: IPU

This is the idea introduced in the google pixel family with their Pixel Visual Core that allows data processing alongside the CPU. They use multiple small ASIP and use their locality on the die to exchange information between each other. It is a fast paced processing where data is consumed and sent to the next processor.

They arange multiple Processing Elements (PE) next to each other and hardwired them all together. They are composed of ALU's and MAC units and can be programmed and sent to the next PE. The dawn of NPU…

In short, the goal is to exploit **parallelism** and functional pipelining. We try to provide more programmability without losing in efficiency.

2.1.2 GPU applications, architecture and operation

> Repeat of Computer Architecture, Summary here.

2.1.2.1 Mobile Here we will try to maximize throughput under power and area limitation. We will also use extensively lower precision data types that can relieve stress and increase throughput while minimizing its impact on the output quality.

We will more and more see tight integration with the CPU and GPU with shared coherent memory space. More and more programmability with CUDA or openCL. Anything that can be parallelized is interesting to run on a GPU.

2.2 Deep learning algorithms

> **Definition of AI**
>
> Any program that mimics human intelligence. A program that can sense, reason, act and adapt

2.2.1 What is deep learning?

- **Machine learning**: is a subset of AI and it is an *algorithm that is able to learn from data*.
  - **Deep learning**: a representational machine-learning using multi-layered neural networks (NN)

The idea of a deep-learning network is one that has many layers and the computer truly learns on its own without much human interaction or tuning. It learns its own representation of the world or task he must accomplish.

2.2.2 From neural nets to deep neural nets

A basic neural network is composed of many **nodes** which forms a **layer**. They are often *fully-connected* to the next layer. The NN will adjust its weight based on algorithm during the training phase. Weight represents how much the information of one node will be forwarded to the next one. The AI can also tweak the **bias** to adjust its performance. Finally, to reduce the "*fuzziness*" we use some **activation functions**.

$$o_n^l = \sigma\left(\sum_m w_{m,n}^l \cdot o_m^{l-1} + b_n^l\right)$$

Table 1: Various σ activation function

Name	Function	Good in math	Easy in HW
Sigmoid	$\frac{1}{1+e^{-x}}$	**V**	**X**
Tanh	$tanh(x)$	**V**	**X**
ReLU	$max(0, x)$	**X**	**V**
Leaky ReLU	$max(0.1x, x)$	**X**	**V**
Maxout	$max(w_1^T x + b_1, w_2^T x + b_2)$	**X**	**V**
ELU	$x \geqslant 0; x$ else $\alpha(e^x - 1)$	**X**	**V**

By using labeled data, we can train the AI to accomplish a task. Using the AI to do for example pattern recognition is called *inference*.

2.2.3 Classes of deep neural networks:

2.2.3.1 DNN In the simple example of NN, we had a vector as an input. But, we can also have an image (matrix) as the input which we can apply the same pattern. This time, we will use a matrix notation to realize 2D-2D operations; a fully connected layer looks like:

$$o_{nx,ny}^l = \sigma\left(\sum_{mx,my}^{M,M} w_{mx,my,nx,ny}^l \cdot o_{mx,my}^{l-1} + b_n^l\right)$$

The matrices are quite large and this can be detrimental sometimes as one result is based on the full image.

2.2.3.2 CNN We are no longer fully connected, it is a sparse matrix. It has better convergence and faster training.

$$o_{nx,ny}^l = \sigma\left(\sum_{fx,fy}^{FX,FY} w_{kx,ky}^l \cdot o_{xn+kx,ny+ky}^{l-1} + b_n^l\right)$$

This will avoid to have a M matrix but rather a small kernel that is sliding over the matrix and of size $Fx \cdot Fy$. We can go further and perform some tensor kernel to apply the same or different kernel on multiple inputs. Each inputs can influence the same output. A kernel is of size $Fx \cdot Fy \cdot C$.

$$o^l_{nx,ny,k} = \sigma\left(\sum_{fx,fy,c}^{FX,FY,C} w^l_{kx,ky,c} \cdot o^{l-1}_{xn+kx,ny+ky,c} + b^l_n\right)$$

This operation requires 7 nested for loops. Bad news for software engineer, good news for hardware engineer has it opens the door for massive parallelism.

To avoid the size to get too big, we often finish by a **Pool and normalization** layer to "compress" the info. Finally we inject it in a classic fully connected NN to output a scalar result.

There are many architectures and flavor of CNN, each with their pros and cons, trying to solve different problems.

DO THE DEPTHWISE POINTWISE CONV EXPLANATION

2.2.3.3 Transformer and LLM The secret sauce of current LLM's is the use of the **attention** mechanism. It is an algorithmic representation of linguistic property of words and sentences. It will connect words between each other regarding the context which allows to not only process one word but also its context surrounding it. The maximum path length is only $\mathcal{O}(n)$ as the information is already embedded in the word after the attention process.

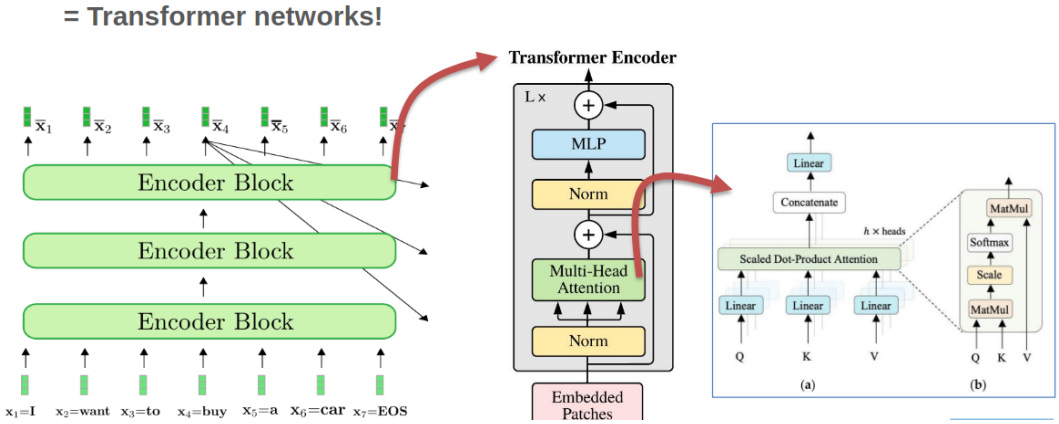

Figure 3: Transformers encoder

The query is for one word we want to inspect, the keys are the result of the query. This form a linear transformation which will tweak the high-dimensional embedding space to "distort" and bring words that are alike closer together. Finally we use the value matrix that will realize a linear transformation to help and find the next most probable word.

$$softmax(\frac{\underline{x}Q_w * XK_w^T}{\sqrt{100}}) * XV_w$$

Multiplication can be quite annoying but with efficient hardware, it can be accelerated. The real bummer here is the $softmax$ function:

$$softmax = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_i}}$$

Where x_i are values. The issue is that it can only be computed after processing all of the data. It is also a non-linear function that cannot be realized in one single pass. This means that we must re-access value from the cache which is slow and **not** energy efficient. Same story for the $norm = \frac{value_{original} - \mu}{\sigma}$.

After doing this pre-fill and encoding, we want to generate some new tokens. This is handled by the decoder block.

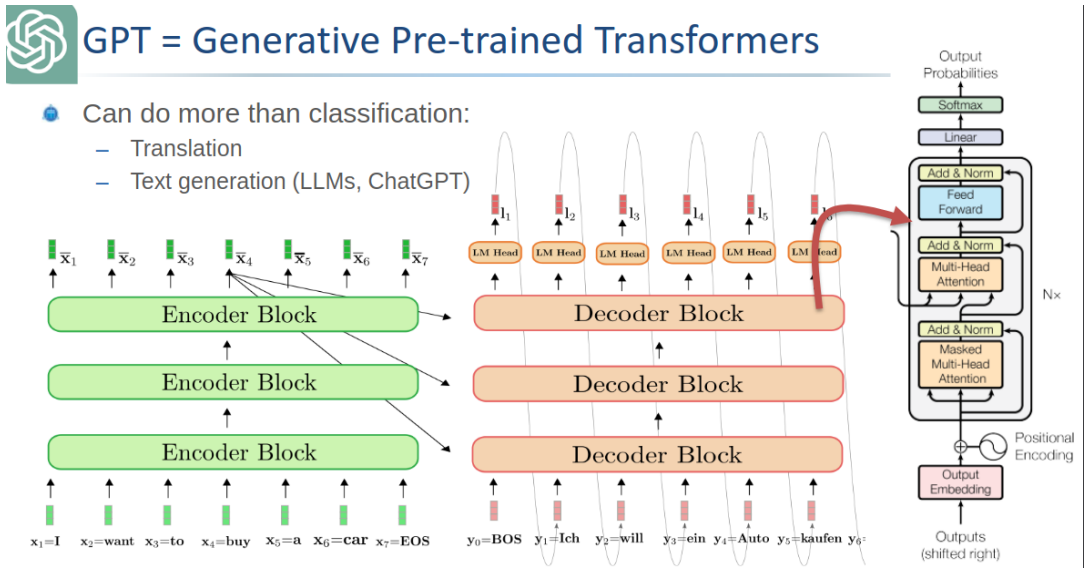

Figure 4: Full architecture

Those matrix multiplications are quite repetitive, we can thus save the result in what we call **KV cache**. We only append the new token and perform some vector matrix computation instead of the full matrix matrix computation.

- Prefill: compute limited
- Decode: memory limited

On a side note, most LLM's are now *decoder-only* but we still have this prefill stage.

3 Lecture 3: Efficient Deep Inference: Hardware Bottlenecks & Parallelization

3.1 Baseline and hardware challenges

The two main efficiency metrics are:

- Latency: time per inference, time to get the first token
- Throughput: inferences per second

We often represent operations per second as TOPs - 10^{12} operations per second. When batching (running multiple queries at once) we gain in efficiency and it provides better throughput as we can reuse the weight of layers loaded in RAM for other users at the same time.

We also look at the energy and power. Where, TOPs/W (or TOP/J) matters depending of the scenario (cooling, embedded, infrastructure, … constrained)

With Moore's law, we should get twice the efficiency roughly every 2 years. The issue is that AI's complexity is getting more and more complex at a higher rate leading to a massive gap between computation abilities and needs.

We can fight at multiple font:

- TP/Latency: $time/op \cdot 1/utilization \cdot ops/inf$
- Energy: $energy/op \cdot 1/utilization \cdot ops/inf$

3.1.1 Metrics for execution efficiency

Matrix multiplications are needed in prefill stage. We call this operation a General Matrix Multiplication or GeMM. This type of optimized operations are implemented in the BLAS library that ensure fast and smart computation. A CPU has small **Processing Element (PE)** with specific datapath to accelerate such calculations.

The main drawback is the recurrent movements of data from on-chip to off-chip. A typical NPU will try to optimize those steps. Sadly, we will hit the memory wall which will lead to unavoidable latency and energy constraints.

One solution is to use lower precision operations to reduce the energy cost. Thankfully, energy savings is linear with the precision but the quality of the LLM isn't, especially if we use smart algorithm.

3.1.2 A tale of two rooflines

The roofline diagram we are familiar with is the $AI - TOPs$ one. Where we plot the Arithmetic Intensity as Operations per bytes of memory access. We have one horizontal line where there is the maximum compute in TOPs N_{op} and a diagonal line that is $AI \cdot BW$ depicting the bottleneck of the memory access. The ideal operating point is where those two line crosses as we are using the maximum of memory and compute. Otherwise the maximum attainable performance is $min(AI \cdot BW, N_{op})$.

But we can also have the **energy roofline**. We wil have the horizontal line being the minimum for an operation (without the transfer and other operations) and the the energy per DRAM access that is AI/E_{DRAM} access. We have the attainable efficiency as

$$\text{Attainable efficiency} = \frac{1}{E_{comp} + E_{mem}/AI}$$

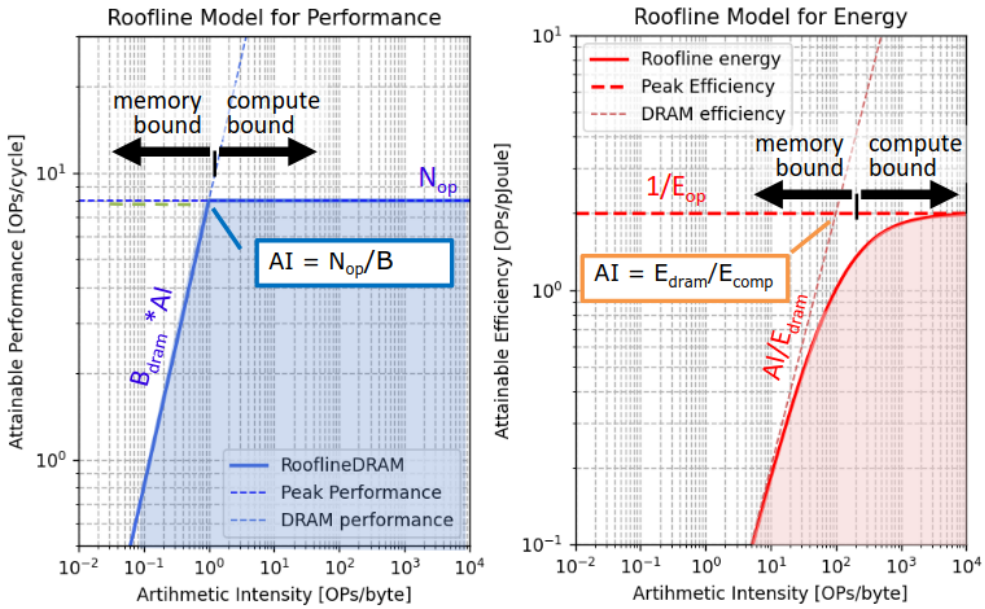

Figure 5: The rooflines diagram

We can see that the optimal performance point may not be the most energy efficient one! It all depends on our goals and seeing the large scale AI infrastructure we prefer to be slightly compute bound to avoid excess energy overhead.

3.2 xPU processor enhancements for AI

As explained earlier, the extra transistors we are gaining thanks to Moore's law need to be used to something. That's why we choose to do specific ASICS block or xPU. They all rely on one of the 5 tricks explained in the coming subsections.

Every trick here will try to push some limits (AI, BW, ...) to further improve the performances.

3.2.1 Parallelization (spatial unrolling optimization)

3.2.1.1 Parallelization in CPU and GPU cores Sadly, simply parallelizing work won't reduce the AI or increase it. We need to also push the peak performance or else no gain.

This is why we are interested in SIMD (parallel MAC in FP), Super-scalar (parallel load/store and compute) and also multi-core parallelism.

A good solution that is often employed is **fused multiply add** to avoid the extra load/store. Multiple inputs for one output, we almost divide memory access by 2.

Basic GPU's do not have this fused MAC in the classic CUDA cores, only massive parallelized MAC.

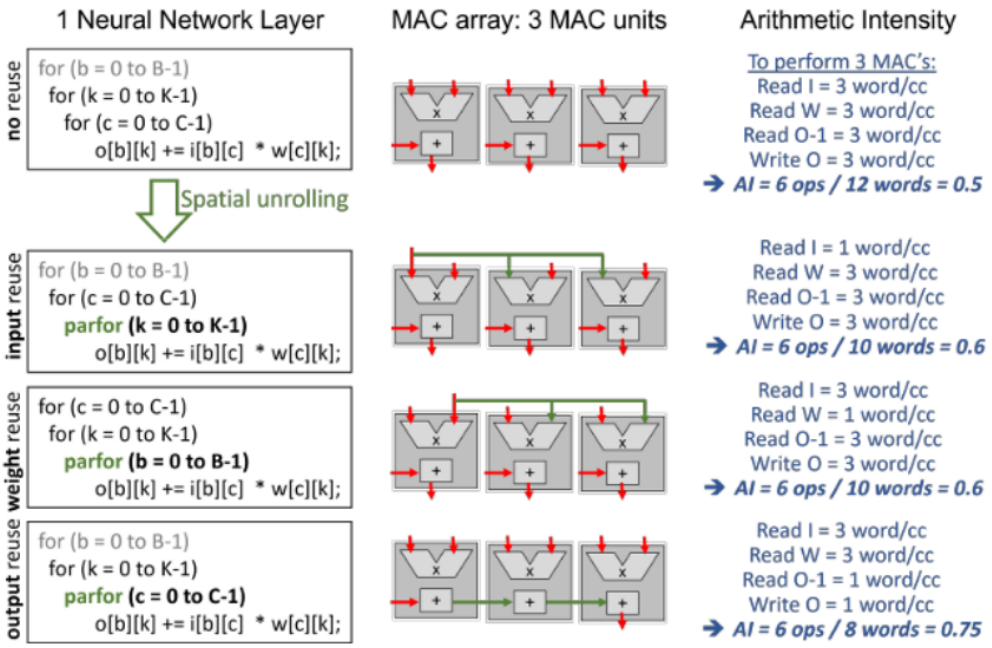

Figure 6: The goal is to reuse data to reduce AI

3.2.1.2 Advanced spatial data reuse in NPU's & Tensor Cores Of course, if we could unroll and reuse everything it would be ideal but the values are getting quickly out of hand. For example, with $B = 9; C = 4; K = 8$ where $B \times C$ and $C \times K$ are the inputs and weights dimensions respectfully. We thus need $B \cdot C \cdot K = 288$ MAC to do all the computations all at once. For the amount of access we have $K \cdot C + B \cdot C + B \cdot K = 8 \cdot 4 + 9 \cdot 4 + 9 \cdot 8 =$.

But we can't have such high number of mac even for those simple architectures. We often have only $100 - 1000$'s MAC in an accelerator. We will smartly fold convolutions on multiplier array. The main optimization criterion will be to optimize towards minimal memory access.

A good way is to mix spatial and temporal unrolling. For example do sub-vector sub-vector multiplication. The compiler can also optimize the loops and re-organize (tiling, ...), this is **dataflow optimization**.

```
for (b = 0 to B-1); // for each image in the batch
  for (k = 0 to K-1); // for each output channel
    for (c = 0 to C-1); // for each input channel
      o[b][k] += i[b][c] * w[c][k];

// Compiler optimization - tiling + spatial optimization
for (b2 = 0 to B/S-1); // for each image in the batch
  for (k = 0 to K-1); // for each output channel
    for (c = 0 to C-1); // for each input channel
      parfor (b1 = 0 to S-1); // for each image in the batch
        o[b2*S+b1][k] += i[b2*S+b1][c] * w[c][k];
```

In this code example, we are dong *weight reuse* as we the weight are reused (the indices are not impacted by a parfor loop so only 1 will be loaded at a time). If we insert the parfor in any other lines we will do some input or output reuse.

Table 2: Comparison of techniques, S = spatial reuse factor

	Weight reuse	Input reuse	Output reuse $\rightarrow$ FMA
Weight BW	1	S	S
Input BW	S	1	S
Output BW	$S+S$	$S+S$	$1+1$
AI	$2S/(3S+1)$	$2S/(3S+1)$	$2S/(2S+2)$

3.2.1.3 State of the Art examples

The real magic sauce of NVIDIA is their tensor cores...

3.2.2 Stationarity (temporal unrolling optimization)

3.2.3 Operator fusion

3.2.4 Quantization

3.2.5 Sparsity

4 Paper to Read

4.1 Lecture 1

> **Paper to read**
>
> *Paper1*: A New Golden Age for Computer Architecture J. Hennessy AND D. Patterson
>
> *Paper2*: Apple M1: Ditching x86 A. Frumusanu
>
> *Paper3*: Paper2: Apple M1 deep dive: Micro-architecture (pg2) Anandtech, Andrei Frumusanu; *link related to the article*

4.2 Lecture 2

> **Paper to read**
>
> *Paper 1*: Attention is all you need sec. 1-5

4.3 Lecture 3

> **Paper to read**
>
> *Paper 1*: How to keep pushing ML accelerator performance? Know your rooflines!

5 Questions

5.1 Lecture 1

1. *What is Dennard's law and how is it linked to the evolution of computer architectures over the last 30 years? Describe the different phases we see in this evolution, and the architectural consequences. Illustrate this with examples from processors discussed in class and the papers to be read.*

> **To be answered**

2. *Why do Henessey and Patterson say it is a New Golden Age for computer architectures, and which opportunities should be exploited in this Golden Age, according to them? (See L1_Turing.pdf)*

> **To be answered**

3. *Discuss the different types of processors present in a modern embedded system (like iPhone's, smart glasses, Tesla cars, or…). In what sense do they differ? Why are they all there? How do they exploit area for efficiency? (after L8: How would they exchange data and processing jobs?)*

> **To be answered**

4. *Apply the different concepts from this class to the Apple M1 processor. Which "area for efficiency" techniques do they exploit and why? What is unique about the FireStorm cores. (See also paper L1_Apple.pdf)*

> **To be answered**

5. *What are the reasons for the going to multi-core CPU's, first homogeneous and later heterogeneous? How are the micro-architectures of the CPU's different/similar, and how are they exploited? Does this offer energy efficiency and/or carbon footprint benefits?*

> **To be answered**

5.2 Lecture 2

6. What are ISP's and IPU's? Explain the evolution in their architecture, using example processors to illustrate the trends. What are the main characteristics that distinguish the Google Pixel IPU from a CPU and from the Hexagon processor?

> **To be answered**

7. Explain the workload of embedded rendering tasks and how this leads to new GPU processing architectures beyond CPU's. What are the main characteristics that distinguish GPU's from CPU's? How do mobile GPU's differ from cloud GPU's?

> **To be answered**

8. What is the difference between a fully connected, a convolutional neural network layer and a depthwise separable layer. Which one is considered more hardware efficient, and why?

> **To be answered**

9. What is the attention mechanism in transformers, and what operations does it require in prefill and decode? what are its benefits and downsides compared to convolutional networks. (Also use L2_Attention.pdf)

> **To be answered**

10. (After having studied L3-5) How can certain types of neural network layers be more efficient from an algorithmic point of view and at the same time be more inefficient from a HW point of view?

> **To be answered**

6 License

6.1 Modification

To contribute to this work or any other one from this project please find more information at the Github repository.
